# Project 1 - Set Up Linux
## Project Report

**COMP7500 - Advanced Operating Systems**

**Chris HInkson**

# I. Table of Contents

**I. Table of Contents** ..... 2
**II. Project Introduction** ..... 3
**III. Installing Ubuntu with VMware Fusion** ..... 4
1. Operating System Installation ..... 4
2. Package Installation ..... 5
**IV. Getting To Know The System** ..... 6
1. CPU Frequency ..... 6
2. Cache Size ..... 6
3. Memory Size ..... 6
4. List of PCI Devices ..... 7
5. Hard Drive ..... 7
6. Network MAC Address ..... 7
7. Link Speed ..... 7
8. Devices Generating Interrupts ..... 8
**V. Trying Out System Utilities** ..... 10
1. Creating the simple.c program with VIM ..... 10
2. Compiling the simple.c program with the GCC compiler ..... 10
3. Examining system libraries used by the simple.c program ..... 11
4. Debugging the simple.c program with the GDB debugger ..... 11
**VI. Challenges and Lessons Learned** ..... 12
1. My first time using VMware ..... 12
2. Different operating system, different architecture, different packages ..... 13
3. Virtual machines don't quite have the same information specs ..... 13
4. Correcting the compile command ..... 14
5. Working with the .gdbinit file ..... 14
**VII. Conclusion** ..... 15

# II. Project Introduction

This project is an introductory project to install and become familiar with a Linux operating system. For this project, I have chosen to use Ubuntu (specifically, Ubuntu 25.10), since CentOS is no longer being maintained.

Once Ubuntu was installed, I then got to know the system and tried out some of the system utilities to further familiarize myself with working in the Linux environment. This part of the project was mostly completed by following the provided project instructions, with minor deviations to further investigate specific features.

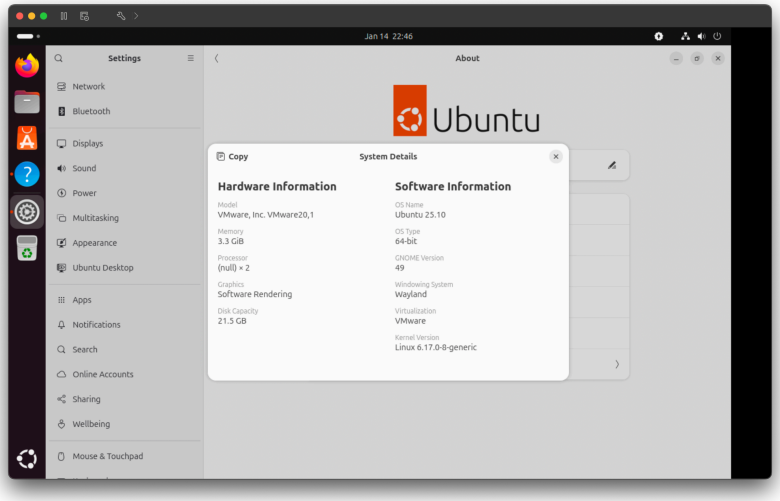
# III. Installing Ubuntu with VMware Fusion

## 1. Operating System Installation

The first step I took when installing my Linux system was deciding which hypervisor I would use to manage my virtual machine. As I use a Macbook for development and academic projects, I would need such a hypervisor in order to run a Linux system in a virtual machine on my Macbook. I chose to use VMware as it has an active community and modern feel for Mac (specifically, VMware Fusion, which is the Mac counterpart to VMware Workstation on Windows/Linux).

Once VMware Fusion was installed, I then decided which Linux distribution I would use for this project. From the two possible options, being CentOS and Ubuntu, I chose to install Ubuntu as it has become a very popular choice throughout the Linux community and is still actively maintained, unlike CentOS. I personally have no previous experience with using either distribution directly, although I have had some light experience using Linux Mint, which is based on Ubuntu.

To install Ubuntu using VMware, I first downloaded the appropriate Ubuntu 25.10 ISO file from the Ubuntu Desktop downloads page for ARM 64-bit architecture. This specific variant of the ISO file is needed to ensure that the system will correctly work on my Macbook. Once downloaded, I then simply used VMware Fusion to create a new virtual machine with the ISO file. After allowing the virtual machine time to initialize and start up, I was able to confirm that Ubuntu was successfully installed as shown in the below image.



## 2. Package Installation

The next step of installing Ubuntu was installing several packages to prepare the system. All of the following mentioned script files are provided alongside this report.

First, I installed the vmware-tools package, which enables features within VMware such as clipboard and shared folders. The process of installing this is shown in the `scripts-installations-VMtools.script` file.

Next, I installed VSCode to allow for easier development and file viewing. The process of installing VSCode with a .deb package is shown in the `scripts-installations-VSCode.script` file.

Finally, I installed all of the mentioned packages in the assignment instructions. This includes gcc, emacs, gdb, etc. These installations are shown in the `scripts-installations-AssignmentPackages.script` file.

# IV. Getting To Know The System

The next phase of the project deals with getting to know the new Linux operating system. To do this, I perform several terminal commands, which are saved to several script files alongside this report. Note that there is a `scripts-systeminformation-all.script` file, which contains all of the initial command runs, along with individual script files for each command used. From running these commands, the following information was found:

## 1. CPU Frequency

There is no reportable CPU frequency as this is a virtual machine. Any utility that reports a CPU frequency, such as `lscpu`, specifies it as 0.0Ghz.

## 2. Cache Size

From the `more /proc/meminfo` command, it can be seen that there is `1357708 kB` for the cache.

## 3. Memory Size

From the `more /proc/meminfo` command, it can be seen that there is `3465504 kB` total memory, with `234108 kB` free memory.

## 4. List of PCI Devices

While on a virtual machine, the PCI devices are listed as follows with the `lscpi` command:

- Host bridge: VMware Device 1976
- System peripheral: VMware Virtual Machine Communication Interface
- VGA compatible controller: VMware SVGA II Adaptor (Fusion)
- Several PCI bridges
- USB controller: VMware USB1.1 UHCI Controller
- Audio device: VMware HD Audio Controller
- USB controller: VMware USB2 EHCI Controller
- SATA controller: VMware SATA AHCI Controller
- Ethernet controller: Intel Corporation 82574L Gigabit Network Connection
- USB controller: CMware Device 077a
- Volatile memory controller: VMware NVMe SSD Controller

## 5. Hard Drive

While I ran several commands to extract information about the hard disk, the following excerpt from `sudo fdisk -l` provides a nice look at what is available on the virtual machine:

Disk /dev/nvme0n1: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VMware Virtual NVMe Disk
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 4BADA8CB-6F21-4876-8EF7-BBFE8F70B161

## 6. Network MAC Address

From `ifconfig`, I can see that my MAC address is `00:0c:29:7b:c6:7d`.

## 7. Link Speed

Although I am inside a virtual machine, `ethtool enp2s0` reports a link speed of 1000Mb/s.

## 8. Devices Generating Interrupts

With `/proc/interrupts`, I am able to find the following devices that generate interrupts:

```
       CPU0       CPU1
 10:   414706     425401    GICv3  27 Level     arch_timer
 12:        0          0 GICv2m-PCI-MSI-0000:00:15.0   0 Edge      PCIe PME
 13:        0          0 GICv2m-PCI-MSI-0000:00:15.1   0 Edge      PCIe PME
 14:        0          0 GICv2m-PCI-MSI-0000:00:15.2   0 Edge      PCIe PME
 15:        0          0 GICv2m-PCI-MSI-0000:00:15.3   0 Edge      PCIe PME
 16:        0          0 GICv2m-PCI-MSI-0000:00:15.4   0 Edge      PCIe PME
 17:        0          0 GICv2m-PCI-MSI-0000:00:15.5   0 Edge      PCIe PME
 18:        0          0 GICv2m-PCI-MSI-0000:00:15.6   0 Edge      PCIe PME
 19:        0          0 GICv2m-PCI-MSI-0000:00:15.7   0 Edge      PCIe PME
 20:        0          0 GICv2m-PCI-MSI-0000:00:16.0   0 Edge      PCIe PME
 21:        0          0 GICv2m-PCI-MSI-0000:00:16.1   0 Edge      PCIe PME
 22:        0          0 GICv2m-PCI-MSI-0000:00:16.2   0 Edge      PCIe PME
 23:        0          0 GICv2m-PCI-MSI-0000:00:16.3   0 Edge      PCIe PME
 24:        0          0 GICv2m-PCI-MSI-0000:00:16.4   0 Edge      PCIe PME
 25:        0          0 GICv2m-PCI-MSI-0000:00:16.5   0 Edge      PCIe PME
 26:        0          0 GICv2m-PCI-MSI-0000:00:16.6   0 Edge      PCIe PME
 27:        0          0 GICv2m-PCI-MSI-0000:00:16.7   0 Edge      PCIe PME
 28:        0          0 GICv2m-PCI-MSI-0000:00:17.0   0 Edge      PCIe PME
 29:        0          0 GICv2m-PCI-MSI-0000:00:17.1   0 Edge      PCIe PME
 30:        0          0 GICv2m-PCI-MSI-0000:00:17.2   0 Edge      PCIe PME
 31:        0          0 GICv2m-PCI-MSI-0000:00:17.3   0 Edge      PCIe PME
 32:        0          0 GICv2m-PCI-MSI-0000:00:17.4   0 Edge      PCIe PME
 33:        0          0 GICv2m-PCI-MSI-0000:00:17.5   0 Edge      PCIe PME
 34:        0          0 GICv2m-PCI-MSI-0000:00:17.6   0 Edge      PCIe PME
 35:        0          0 GICv2m-PCI-MSI-0000:00:17.7   0 Edge      PCIe PME
 36:        0          0 GICv2m-PCI-MSI-0000:00:18.0   0 Edge      PCIe PME
 37:        0          0 GICv2m-PCI-MSI-0000:00:18.1   0 Edge      PCIe PME
 38:        0          0 GICv2m-PCI-MSI-0000:00:18.2   0 Edge      PCIe PME
 39:        0          0 GICv2m-PCI-MSI-0000:00:18.3   0 Edge      PCIe PME
 40:        0          0 GICv2m-PCI-MSI-0000:00:18.4   0 Edge      PCIe PME
 41:        0          0 GICv2m-PCI-MSI-0000:00:18.5   0 Edge      PCIe PME
 42:        0          0 GICv2m-PCI-MSI-0000:00:18.6   0 Edge      PCIe PME
 43:        0          0 GICv2m-PCI-MSI-0000:00:18.7   0 Edge      PCIe PME
 46:    24480          0 GICv2m-PCI-MSIX-0000:0a:00.0   0 Edge      xhci_hcd
```

```
 49:        0        0   GICv3  32 Level     ehci_hcd:usb3
 50:        0        0   GICv3  23 Level     arm-pmu
 52:     1525        0 GICv2m-PCI-MSI-0000:01:03.0   0 Edge      ahci[0000:0
1:03.0]
 53:       29        0 GICv2m-PCI-MSIX-0000:12:00.0   0 Edge      nvme0q0
 54:    59876        0 GICv2m-PCI-MSIX-0000:12:00.0   1 Edge      nvme0q1
 55:        0    58100 GICv2m-PCI-MSIX-0000:12:00.0   2 Edge      nvme0q2
 56:       10        0 GICv2m-PCI-MSIX-0000:00:0f.0   0 Edge      vmwgfx
 57:        0        0 GICv2m-PCI-MSIX-0000:00:0f.0   1 Edge      vmwgfx
 58:        0        0 GICv2m-PCI-MSIX-0000:00:0f.0   2 Edge      vmwgfx
 59:        3        0 GICv2m-PCI-MSIX-0000:00:0f.0   3 Edge      vmwgfx
 60:        0        0 GICv2m-PCI-MSIX-0000:00:0f.0   4 Edge      vmwgfx
 61:        0        0 GICv2m-PCI-MSIX-0000:00:0f.0   5 Edge      vmwgfx
 62:      537        0 GICv2m-PCI-MSIX-0000:00:00.7   0 Edge      vmw_vmci
 63:        0        0 GICv2m-PCI-MSIX-0000:00:00.7   1 Edge      vmw_vmci
 64:     1078        0 GICv2m-PCI-MSIX-0000:00:00.7   2 Edge      vmw_vmci
 65:    19887        0 GICv2m-PCI-MSIX-0000:02:00.0   0 Edge      enp2s0-rx-
0
 66:    18329        0 GICv2m-PCI-MSIX-0000:02:00.0   1 Edge      enp2s0-tx-
0
 67:        1        0 GICv2m-PCI-MSIX-0000:02:00.0   2 Edge      enp2s0
 68:     1955        0 GICv2m-PCI-MSI-0000:01:01.0   0 Edge      snd_hda_int
el:card0
IPI0:   114829   115678       Rescheduling interrupts
IPI1:   474445   472951       Function call interrupts
IPI2:        0        0       CPU stop interrupts
IPI3:        0        0       CPU stop NMIs
IPI4:        0        0       Timer broadcast interrupts
IPI5:       54       50       IRQ work interrupts
IPI6:        0        0       CPU backtrace interrupts
IPI7:        0        0       KGDB roundup interrupts
```

# V. Trying Out System Utilities

The final phase of the project involved trying out the system utilities such as the VIM file editor, GCC compiler, system libraries, and GDB debugger. To familiarize myself with these tools, I created the program as specified in the assignment instructions to find the average of the square roots of an array of numbers.

## 1. Creating the simple.c program with VIM

The first step I took was to create the simple.c program with the VIM command line editor. I first opened VIM with the command `vim simple.c` and then moved to insertion mode with the `i` key. I then created my program with a hard-defined list of 10 numbers and a simple algorithm to iterate through them and add together their square roots. The program then divides this sum by the number of elements in the array (in this case, 10) to obtain the average square root of the array of numbers. Once done, I then exited VIM with `:wq`, creating the file and saving it with my entered file contents. This program is provided alongside this project in the `simple.c` file.

## 2. Compiling the simple.c program with the GCC compiler

Compiling the program was very easy with GCC. First, I made sure that GCC was ready with the `gcc -v` command to check the version. Then, I compiled the program with `gcc -g -o simple simple.c -lm`. Note that this command is slightly different than the one provided in the assignment instructions, which I discuss in further detail later in this report. The compilation of my program is shown in the `scripts-development-compile.script` file located alongside this report.

## 3. Examining system libraries used by the simple.c program

To examine the system libraries used by the program, I used the `ldd` command. This allowed me to observe the following system libraries:

libm.so.6 => /lib/aarch64-linux-gnu/libm.so.6 (0x0000e7a4c69e0000)
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6 (0x0000e7a4c67e0000)

This indicates that my program is using the math and libc libraries, which is expected based on the program contents. The use of the `ldd` command is shown in the `scripts-development-ldd.script` file located alongside this report.

## 4. Debugging the simple.c program with the GDB debugger

To practice debugging my program with the GDB debugger, I used the several actions specified in the assignment instructions.

First, I made the `.gdbinit` file as specified in the assignment instructions. This file allowed me to automatically set breakpoints and see register information. Note that I did have to slightly modify this file to work properly, as discussed later in this report.

I then started the debugger with the `gdb ./simple` command. I could then use the `r` command to run the program, the `n` command to move to the next line, the `s` command to step into function calls, and the `c` command to continue. My practice with the GDB debugger with these commands is shown in the `script-development-gdb.script` file located alongside this report.

# VI. Challenges and Lessons Learned

Most of the challenges faced in this project related to re-familiarizing myself with the Linux operating system (and specifically, with the Ubuntu operating system) and the command line utilities for C-based development.

## 1. My first time using VMware

As this was my first time using VMware on my machine, I had to learn how to properly setup and configure a virtual machine using it. Although I had used VirtualBox a long time ago, VMware was still relatively new to me.

Luckily, thanks to the provided guide for installing VMware and Ubuntu, I was able to install and set up my environment fairly quickly. As discussed in the guide, VMware makes virtual machine creation fairly straightforward, as all I needed was a .iso file and to click a few buttons.

One further step I had to complete was installing VMware Tools. While I knew the features existed, I did not know exactly how to enable things like clipboard sharing and mounting folders from my computer. Luckily, hints within VMware Fusion made researching the features easier for me to find VMware Tools. Installing this utility was very easy as online resources (such as this Broadcom website article) provide clear walkthroughs of how to set it up and use it. Without VMware Tools, this project would likely have taken much longer.

## 2. Different operating system, different architecture, different packages

One challenge that I anticipated ahead of time was the troubleshooting I would have to complete when using a new operating system within a virtual machine and attempting to install certain applications and packages. For instance, some of the packages mentioned in the assignment instructions or online guides do not exist for Ubuntu. Additionally, I had to account for the specific architecture provided by the virtual machine when installing packages, and encountered a few instances where I attempted to install packages for the wrong architecture. Luckily, command line errors / warnings were very helpful, and it was quite easy to find when I was attempting to do this. I quickly moved past this challenge and ensured that my system still met all requirements for completing this project.

## 3. Virtual machines don't quite have the same information specs

Although this did not prevent anything, and is certainly expected, one slight challenge is that virtual machines do not provide certain specifications or measurements relating to hardware. For instance, the CPU frequency and link speed information is not something real to a virtual machine - this kind of information is sometimes hidden or spoofed when using a virtual machine. Again, this was expected, but it was something that had to be acknowledged when reporting this information for the project.

## 4. Correcting the compile command

One challenge that did require some further investigation was the gcc compilation command. The assignment instructions originally gave the command as `gcc -g -lm -o simple simple.c`, although after trying it, the command did not compile my program correctly and gave errors. After looking into the cause of the errors, I found that instead I would need the `-lm` flag to come after the reference to my program. Luckily, this was a quick fix, as the command `gcc -g -o simple simple.c -lm` successfully compiled my program. I found the following StackOverflow posts to be particularly helpful in researching this issue:

- What is the meaning of -lm in GCC? - Stack Overflow
- gcc/g++ parameter order - Stack Overflow

## 5. Working with the .gdbinit file

I did encounter some difficulty in using the .gdbinit file.

First, the file on its own was not being properly used by GDB when attempting to debug the program. Luckily, command line warnings (which give the same information as shown on the man page) indicated that this was due to gdbinit not being enabled. I was able to fix this by enabling the gdbinit as discussed in the man page for gdb.

Next, actually using the file as provided did not initially work. I traced this error down to the `info registers` line, which essentially resulted in asking for register information before the program was started. To fix this, I added the program run into the gdbinit, allowing for the program to run before asking for register info. I found this article particularly useful for understanding what `info registers` is showing and how it can be used.

# VII. Conclusion

In conclusion, I was able to install Ubuntu with a VMware virtual machine, examine system information of the new operating system, and write a simple program to practice command line utilities. I feel that the project was successful, as I now feel prepared to move onto more advanced projects with this knowledge.